ML4N - Project 8 Report: Patterns in Comments From Reddit Social Network

ALESSIA BIANCOLILLO and GIACOMO PETTINATO

This project investigates the patterns in the Reddit comments using supervised and unsupervised machine learning techniques on a large-scale dataset. For the supervised task, the goal was to predict users' gender from their comments. Three models were evaluated with the best performance obtained by a tuned Random Forest model. For the unsupervised task, K-Means and HDBSCAN were used to group comments based on the words contained in them. The goal was to identify patterns and group comments with similar words. K-Means captured broad but highly imbalanced structures, while HDBSCAN identified small, semantically coherent clusters, although most samples were classified as outliers. Finally, sentiment analysis was performed using a pre-trained sentiment analysis model, showing a predominance of neutral sentiment and limited contribution to improving gender classification.

1 Introduction

In recent years, social networking platforms have become central spaces for online communication, with Reddit being one of the largest online forums. Reddit is structured into topic-specific communities called subreddits, covering a wide range of subjects such as news, gaming, and books. This structure makes Reddit a rich and widely used source of data for research in natural language processing (NLP) and machine learning.

This project uses a large Reddit dataset of approximately 20,000 users and 1.4 million comments. The main objectives are twofold: first, to develop supervised learning models to predict user gender based on textual comment data; second, to apply unsupervised learning techniques to discover patterns and structures in the data without using labels.

To achieve these goals, various preprocessing techniques are applied, including text cleaning, feature extraction using Bag-of-Words and TF-IDF representations, and dimensionality reduction methods. These features are then used to train machine learning models and analyze the dataset structure.

2 Data exploration and pre-processing

The aim of this section is to explore the datasets, clean and standardize the data, convert raw text into numerical features, and apply dimensionality reduction. We worked with three different datasets:

- supervised: contains 296,042 comments used from 5000 users with self reported gender.
- target: contains the gender (0 = male, 1 = female) for each author of the supervised dataset.
- unsupervised: contains 1,107,946 comments from 15000 users without gender labels.

Each comment contains the following features:

- author: the username of who wrote the comment
- subreddit: the name of the subreddit where the comment was posted
- created utc: data and time of when the comment was posted (UNIX time format)
- body: the text of the comment

2.1 Data exploration

2.1.1 Data quality and feature analysis. During data exploration, it has been possible to observe that the supervised dataset contains 5,000 unique authors, while the unsupervised dataset includes 15,000 authors, confirming that the latter is three times larger in terms of users. The number of unique subreddits is comparable across the two datasets, indicating a similar diversity of discussion topics. Both datasets exhibit a high level of diversity in terms of users, content, and topics.

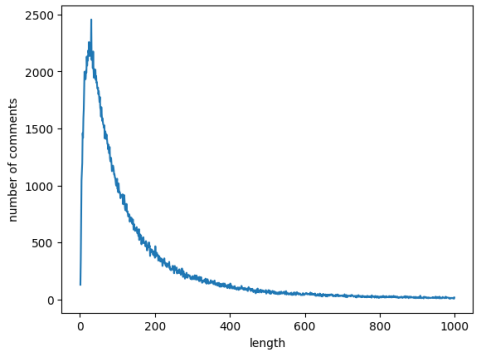
Authors' Contact Information: Alessia Biancolillo; Giacomo Pettinato.

Table 1. Number of unique values for each feature in the supervised and unsupervised datasets

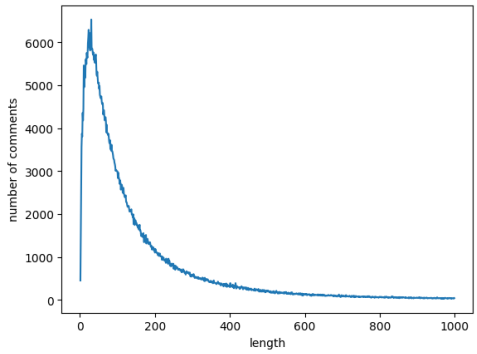
Feature	Supervised Dataset	Unsupervised Dataset
Author	5000	15000
Subreddit	3468	3970
Created_utc	278142	715561
Body	289608	828126

The initial preprocessing step involved removing missing values and duplicate comments. In the supervised dataset, even after deduplication, 2% of comments remained identical across different authors; these were retained as they may capture meaningful correlations between users. The unsupervised dataset originally contained 1,107,946 comments, which reduced to 851,807 after cleaning, with 97% of comments having unique text.

2.1.2 Data distribution analysis. The empirical distribution of comment lengths in both the supervised and unsupervised datasets was initially analyzed without applying any filtering. The presence of a small number of extremely long comments significantly distorted the scale of the distributions, reducing interpretability. To address this issue, a filtered version was introduced by retaining only comments shorter than 1000 characters (Figure 1). In this setting, both datasets exhibit a clear peak between 0 and 200 characters, followed by a rapid decay resembling an exponential distribution. This suggests that user interactions are generally concise, while very long comments occur only rarely.



(a) Supervised dataset (comments < 1000 characters)



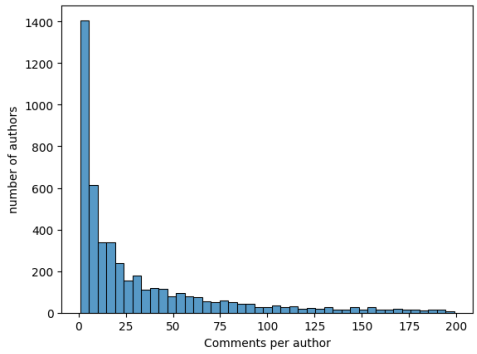
(b) Unsupervised dataset (comments < 1000 characters)

Fig. 1. Empirical distribution of the number of characters per comment for both filtered datasets

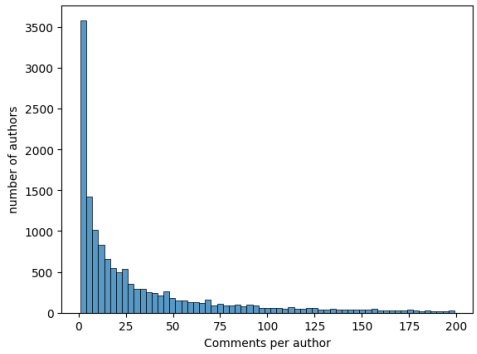
The distribution of comments per author is examined in Figure 2, restricting the analysis to users with fewer than 200 comments in order to mitigate the influence of extreme outliers.

The resulting distribution is strongly right-skewed, indicating that most users contribute only a small number of comments, while there are very few highly active authors. This imbalance is particularly relevant for tasks such as gender classification, especially given that the supervised dataset also exhibits an uneven distribution, with male users producing substantially more comments than female users

Finally, the language distribution of comments in the supervised dataset (Figure 3) was investigated, revealing a strong predominance of English. However, automatic language detection (langdetect) is not always reliable on short, informal Reddit-style texts, where slang and non-standard structures can lead to misclassification or low



(a) Comments per author in supervised dataset



(b) Comments per author in unsupervised dataset

Fig. 2. Histograms of comments per author in both datset

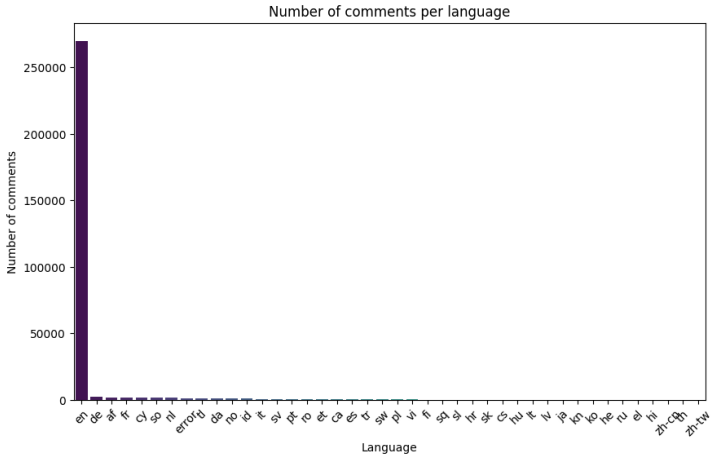


Fig. 3. Distribution of detected languages in the dataset

confidence assignments. For this reason, only comments classified as English with sufficient confidence (over 0.4) were retained, improving the overall consistency of the dataset by reducing noise introduced by the limitations of the language detection model.

2.2 Data cleaning

2.2.1 Standardization of text format. To ensure consistency in the textual data, all comments were first normalized by converting characters to lowercase. In addition, basic text cleaning was applied to remove elements that do not contribute meaningful semantic information, such as URLs and references to subreddits or users. Further cleaning steps were applied to eliminate punctuation, numerical values, and other non-informative elements. These components do not contribute meaningful semantic information and may introduce noise, especially in informal user-generated content such as Reddit comments.

2.2.2 Deletion of stop word. To reduce noise in the textual data, common stop words such as “the”, “a”, and “is” were removed using the NLTK library. These words are very frequent but carry limited semantic information and are unlikely to contribute meaningfully to the learning task.

2.2.3 Text reduction. To reduce lexical variability, lemmatization was applied using the NLTK library. This process converts words to their base form (e.g., “running” → “run”), helping to normalize inflected variants while preserving meaning. Compared to stemming, which produces more aggressive and less interpretable transformations, lemmatization was preferred as it maintains more meaningful and linguistically correct representations of words. This is particularly important in the context of user-generated content, where preserving readability and structure can be beneficial for downstream analysis tasks such as gender classification.

2.3 Text Vectorization

2.3.1 User-Level Aggregation. For the supervised task, all comments written by the same author were grouped and concatenated into a single document, resulting in a dataset where each row corresponds to one user. This representation is more suitable for gender classification, as it enables the model to learn linguistic patterns from the author’s overall writing behavior.

2.3.2 Text conversion. Since machine learning algorithms cannot directly process textual data, the comments were converted into numerical vectors using the Bag of Words (BoW) representation. First, a vocabulary was built from all the words appearing in the corpus. Then, each author document was represented by a vector containing the frequency of every vocabulary term in that document.

2.3.3 BoW vs TF-IDF Representation. While Bag of Words represents each document using raw word counts and treating all words equally, TF-IDF assigns a weight to each word based on frequency. Words that appear frequently in a single document are considered relevant, while words that appear in many documents are considered less informative. Compared to BoW, TF-IDF generally provides a more informative representation, emphasizing words that better characterize each author and and improving performance in classification tasks.

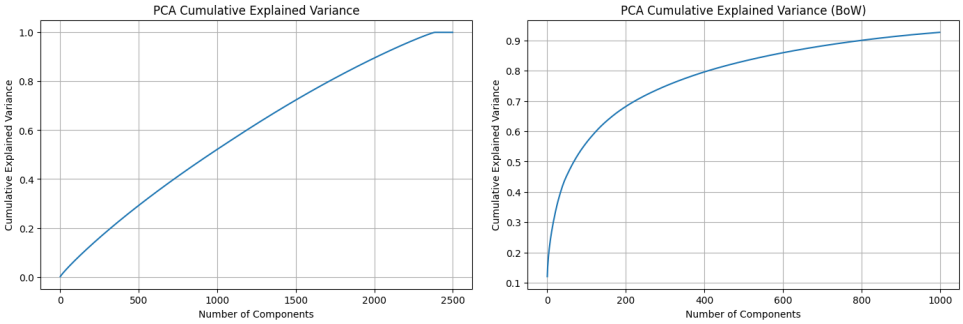
2.4 Reduce complexity

2.4.1 Threshold-based Feature Reduction. To reduce the dimensionality of the feature space, a frequency-based filtering approach was applied using the Bag of Words representation. Words below the 5th percentile and above the 95th percentile were removed to exclude extremely rare and overly frequent terms. This retains more informative mid-frequency words, reducing noise and producing a more compact feature space for both BoW and TF-IDF.

Different percentile thresholds were tested to control filtering intensity, with the chosen values providing the best trade-off between noise reduction and information retention.

2.4.2 Dimensionality Reduction with PCA . PCA can technically be applied to both BoW and TF-IDF representations after converting the sparse matrices into a dense format. However, its use on high-dimensional text data is computationally expensive, as the information is distributed across a very large and sparse feature space. For this reason, the analysis was performed on reduced subsets of the data.

The number of components was selected based on the cumulative explained variance, which measures the proportion of the original information retained after dimensionality reduction, with a target of 90%, meaning that the minimum number of components required to reach this threshold was retained. In practice, PCA does not provide a direct physical or linguistic interpretation, since each component is a linear combination of original features (words) and therefore cannot be associated with specific terms or topics.



(a) TF-IDF - 2500 authors, 2500 components (b) BoW - 4000 authors, 1000 components

Fig. 4. Cumulative explained variance for BoW and TF-IDF under different configurations of users and PCA components.

For TF-IDF, applying PCA on the full dataset with 1,000 components retained only about 30% of the variance, indicating significant information loss. Increasing the number of components was computationally impractical, so the dataset was reduced to 2,500 users; even then, around 2,000 components were required to reach 90% explained variance, as show in 4

For BoW, the analysis was performed on 4,000 users due to memory constraints. In this case, PCA was more effective, as 1,000 components were sufficient to retain about 95% of the variance, indicating that BoW representation is more compressible than TF-IDF.

Therefore, although PCA is applicable, it is not the most efficient method for dimensionality reduction in high-dimensional sparse text representations.

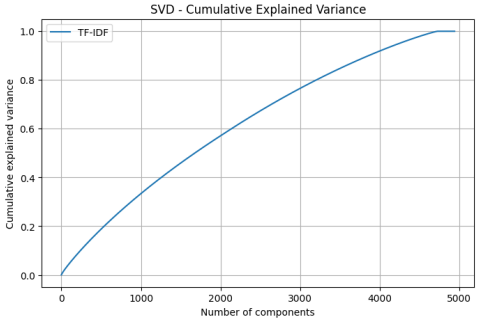


Fig. 5. Truncated SVD with 5000 components

Given PCA's limitations, Truncated SVD was tested instead, as it is specifically designed for high-dimensional sparse matrices reducing memory consumption and making it more suitable for text mining applications.

The results highlighted the different characteristics of the two representations: for BoW, about 600 components were sufficient to retain 90% of the cumulative explained variance, whereas TF-IDF (Figure 5) required roughly 4,500 components to reach the same threshold and was considerably more computationally expensive. This reflects the fact that TF-IDF distributes information across a larger number of features.

3 Supervised learning – classification

The goal of the supervised learning task is to predict the gender of Reddit users based on their textual activity. Since gender information is not natively available on the platform, we use a set of users who reported their gender. By leveraging the textual features extracted from users' comments, we train and evaluate machine learning models to classify users according to their reported gender. In the supervised learning task, to ensure the labels were correct the lower-cased author feature was used as unique identifier. The pre-processed dataset contains 4,943 users instead of the original 5,000 the remaining 57 authors had all their comments removed in the pre-processing stages.

3.1 Split the dataset

The dataset was split into training and test sets using an 70/30 ratio. Since the feature matrices obtained after dimensionality reduction may still contain features with different scales, MaxAbsScaler was applied. The scaler was fitted exclusively on the training set and subsequently used to transform both the training and test sets. This procedure prevents information from the test set from leaking into the training process and ensures a fair evaluation of the models.

3.2 ML model training and evaluation

3.2.1 Model Training and Performance Evaluation. Three machine learning models were evaluated for the gender prediction task: Logistic Regression, Linear SVM, and Random Forest. Additionally a Neural Network was also evaluated in Appendix A. Each model is trained using default hyperparameters and assessed on both training and test sets. All models achieved near-perfect performance on the training set (accuracy $\approx 0.99 - 1.00$), while test performance was substantially lower, indicating strong overfitting. This behavior is likely related to the high-dimensional TF-IDF + SVD representation and the class imbalance present in the dataset. To mitigate overfitting, several strategies can be applied, such as using the BoW rapresentation, using stronger regularization (e.g., tuning the C parameter in linear models), balancing the classes more effectively, or applying cross-validation for more robust model selection. The results can be summarized as follows:

- **Logistic Regression**: lowest test performance (accuracy ≈ 0.40), showing limited predictive power for both classes. (Table 2)
- **Linear SVM**: intermediate test performance (accuracy ≈ 0.49) and the most balanced behavior across classes, although still affected by class imbalance. (Table 4)
- **Random Forest**: highest test accuracy (accuracy ≈ 0.72), but heavily biased toward the majority class and largely unable to identify the minority class. (Table 3)

Overall, Linear SVM provides the most stable and balanced predictions, while Random Forest suffers from strong class bias and Logistic Regression underperforms.

Table 2. Logistic Regression performance

Dataset	Class	Precision	Recall	F1-score	Support	Accuracy
Train	0	0.99	1.00	0.99	2528	0.99
Train	1	0.99	0.96	0.98	932	
Test	0	0.62	0.45	0.52	1084	0.40
Test	1	0.15	0.26	0.19	399	

Table 3. Random Forest performance

Dataset	Class	Precision	Recall	F1-score	Support	Accuracy
Train	0	1.00	1.00	1.00	2528	1.00
Train	1	1.00	1.00	1.00	932	
Test	0	0.73	0.99	0.84	1084	0.72
Test	1	0.24	0.01	0.02	399	

Table 4. Linear SVM performance

Dataset	Class	Precision	Recall	F1-score	Support	Accuracy
Train	0	0.98	1.00	0.99	2528	0.99
Train	1	1.00	0.96	0.98	932	
Test	0	0.66	0.62	0.64	1084	0.49
Test	1	0.12	0.15	0.13	399	

Table 5. Confusion matrices for Logistic Regression, Random Forest, and Linear SVM on the test set

Logistic Regression	Pred 0	Pred 1
Class 0	490	594
Class 1	297	102

Random Forest	Pred 0	Pred 1
Class 0	1068	16
Class 1	394	5

Linear SVM	Pred 0	Pred 1
Class 0	669	415
Class 1	341	58

3.2.2 Tuning of the hyperparamters with cross-validation. To improve the performance of the classifiers, hyperparameter tuning was performed using Grid Search combined with cross-validation, optimizing the macro F1-score to account for class imbalance.

For Logistic Regression, different values of the regularization parameter (C) were explored. The best configuration (C=0.001) improved test accuracy from 0.40 to 0.59 and increased the macro F1-score from 0.36 to 0.42 (Table 6). However, these gains were accompanied by a stronger bias toward the majority class, resulting in lower recall and F1-score for the minority class. Additional experiments with smaller values of C yielded only marginal improvements during cross-validation and did not lead to meaningful gains on the test set. Since stronger regularization would further increase the model's bias toward the majority class, no further tuning of Logistic Regression was pursued. Overall, the results highlight the trade-off between improving overall predictive performance and maintaining good performance on underrepresented classes.

For Linear SVM, several values of C and different loss functions were tested. The best configuration achieved a cross-validation score of approximately 0.41 with (C=0.1 and hinge loss). However, again, this configuration resulted in a model that predicted well the majority class, achieving a relatively high accuracy but performed poorly (0.1 precision) on the minority class. Table 7

The largest improvement was observed for Random Forest. The default model showed severe overfitting, achieving near-perfect performance on the training set while performing poorly on the minority class in the test

Table 6. Classification report for Logistic Regression on the test set.

Class	Precision	Recall	F1-score	Support	Accuracy
0	0.70	0.77	0.73	1084	-
1	0.12	0.09	0.10	399	-
Overall	-	-	-	1483	0.59

Table 7. Classification report for the tuned Linear SVM on the test set.

Class	Precision	Recall	F1-score	Support	Accuracy
0	0.66	0.64	0.65	1084	-
1	0.10	0.12	0.11	399	-
Overall	-	-	-	1483	0.50

set. Hyperparameter tuning reduced this effect by limiting tree complexity through the parameters *max_depth* and *min_samples_split*. The best configuration used 500 trees, a maximum depth of 12, and a minimum split size of 350 samples. This model achieved a test accuracy of 0.76 and a macro F1-score of approximately 0.69, while substantially improving the classification of the minority class, as shown in Table 8.

Table 8. Classification report for the tuned Random Forest on the test set.

Class	Precision	Recall	F1-score	Support	Accuracy
0	0.83	0.84	0.84	1084	-
1	0.55	0.53	0.54	399	-
Overall	-	-	-	1483	0.76

Comparing both predictive performance and computational cost, Random Forest was selected as the best model. Although its training time was substantially higher (approximately 300 seconds) than that of Linear SVM and logistic regression (a few seconds), the improvement in predictive performance was considerable. In particular, Random Forest achieved the highest macro F1-score and provided the best balance between precision and recall across both classes. Therefore, the increased computational cost was justified by the substantial gain in classification performance.

3.3 feature investigation

3.3.1 Different approach: BoW rappresentation. The most interesting observation of this different approach is that the results differ from those obtained with the TF-IDF representation, showing different behaviour across models.

With TF-IDF followed by Truncated SVD, Random Forest is the best performing model (macro F1 $\approx$ 0.69), while Logistic Regression and Linear SVM obtain lower scores. With the reduced BoW representation, Logistic Regression and Linear SVM improve significantly, whereas Random Forest remains strong but without the same dominance as in the TF-IDF setting. This difference is linked to the nature of the representations: TF-IDF highlights discriminative terms and favors tree-based models, while BoW, even after Truncated SVD, preserves more general distributional patterns that better support linear models such as Logistic Regression and Linear SVM. From a quantitative perspective, Random Forest achieved its best accuracy of approximately 74% (500 estimators, max depth = 30, min samples split = 300). However, the result is partly misleading, as the model remains biased toward the majority class and performs worse on the minority class compared to linear models.

Table 9. Classification report for Random Forest on the BoW representation.

Class	Precision	Recall	F1-score	Support	Accuracy
0	0.84	0.80	0.82	1084	-
1	0.51	0.57	0.54	399	-
Overall	-	-	-	1483	0.74

Table 10. Classification report for Logistic Regression on the BoW representation.

Class	Precision	Recall	F1-score	Support	Accuracy
0	0.81	0.63	0.71	1084	-
1	0.38	0.60	0.46	399	-
Overall	-	-	-	1483	0.62

In contrast, Logistic Regression (C = 500) and Linear SVM (C = 50, loss = squared hinge) show a more balanced behaviour across classes. They achieve slightly higher accuracy than the TF-IDF representation and significantly improve minority class detection in terms of recall and F1-score. In particular, Logistic Regression reaches a macro F1-score of approximately 0.59, showing the best trade-off between the two classes in this setting.

Table 11. Classification report for Linear SVM on the BoW representation.

Class	Precision	Recall	F1-score	Support	Accuracy
0	0.81	0.63	0.71	1084	-
1	0.37	0.60	0.46	399	-
Overall	-	-	-	1483	0.62

Regarding the comparison between TF-IDF and BoW, the results confirm that TF-IDF combined with Random Forest remains the best-performing configuration overall, achieving a macro F1-score of approximately 0.69. However, BoW proves to be more effective for linear models such as Logistic Regression and Linear SVM, leading to more stable and balanced performance across classes.

3.3.2 Not Scaling the data . To assess the impact of feature scaling, experiments were repeated on the TF-IDF + Truncated SVD representation without applying MaxAbsScaler. Results show that scaling does not consistently improve performance and may slightly worsen it in some cases.

For Logistic Regression, removing scaling led to a strong improvement: the macro F1-score increased from ≈ 0.42 to ≈ 0.71 (0.85 for class 0 and 0.58 for class 1), and accuracy rose from 59% to 77%.

For Random Forest, performance remained almost unchanged, as expected for tree-based models. Without scaling, it achieved a macro F1-score of ≈ 0.69 (0.83 for class 0 and 0.55 for class 1) and an accuracy of 75%.

In contrast, Linear SVM performed much worse without scaling, with a macro F1-score of 0.42 (0.84 for class 0 and 0 for class 1), effectively predicting only the majority class. This confirms its strong dependence on feature scaling.

Overall, scaling is not universally beneficial: Logistic Regression and Random Forest perform better without it in this setting, while Linear SVM clearly requires it. Among all models, Random Forest remains the most robust across preprocessing choices.

3.3.3 Summary approaches and final pipeline selection. Multiple approaches were investigated for the supervised learning task, focusing on different text representations, preprocessing strategies, and classification models to identify the most effective pipeline.

Two main representations were evaluated: TF-IDF and Bag-of-Words, both combined with Truncated SVD to reduce dimensionality while preserving relevant structure. The impact of feature scaling (MaxAbsScaler) was also analyzed, showing no consistent benefit and in some cases slightly worse performance, indicating that the SVD-transformed space is already well-conditioned.

Three classifiers were tested: Logistic Regression, Random Forest, and Linear SVM, with both default and tuned hyperparameters evaluated via cross-validation using macro F1-score. Overall, Random Forest achieved the best balance between accuracy and macro F1-score, while Logistic Regression and Linear SVM performed competitively mainly in the BoW setting but were more sensitive to preprocessing and representation choices.

The best overall pipeline consisted of TF-IDF + Truncated SVD with a tuned Random Forest (500 estimators, max depth 12, min samples split 350), achieving approximately 0.76 accuracy and 0.69 macro F1-score, representing the most robust configuration.

3.3.4 Adding posting pattern. this final experiment, the best-performing text-based pipeline was extended by adding behavioral features derived from users' posting patterns across subreddits. For each user, a feature vector representing the fraction of posts per subreddit was computed, reduced with Truncated SVD, and concatenated with the TF-IDF + SVD representation.

The augmented feature set was used to train a Random Forest classifier with the previously optimized hyperparameters (500 estimators, maximum depth 12, minimum samples split 350).

The results show a clear improvement over text-only approaches, with an accuracy of 0.83 and a macro F1-score of 0.77, indicating a better balance between classes. In particular, the minority class F1-score increased to 0.66 (vs. 0.54 in previous configurations), reflecting improved detection without significant loss on the majority class. The confusion matrix confirms this improvement, showing fewer false negatives for class 1 and a more balanced overall behavior.

These results suggest that subreddit-level behavioral features add complementary information to textual data: TF-IDF captures linguistic patterns, while subreddit distributions reflect user interests and activity, leading to a more informative feature space.

Table 12. Classification report for Random Forest with TF-IDF + Subreddit features.

Class	Precision	Recall	F1-score	Support	Accuracy
0	0.86	0.91	0.89	1084	-
1	0.72	0.61	0.66	399	-
Overall	-	-	-	1483	0.83

Table 13. Confusion matrix for Random Forest with TF-IDF + Subreddit features.

	Predicted 0	Predicted 1
Actual 0	989	95
Actual 1	155	244

4 Unsupervised learning – clustering

The objective is to cluster Reddit's comments based on the words they contain, to identify patterns and group comments with similar words together. The pre-processing pipeline is similar to one of the supervised task, except for that "author aggregation" is no longer needed, and some pre-processing steps depend on the cluster algorithm used (details in the following subsections).

Two clustering algorithms were employed: K-means and HDBSCAN. K-means was selected for its computational efficiency and scalability, while HDBSCAN was chosen because for its ability to identify outliers.

4.1 K-means

4.1.1 Pre-processing pipeline. The experiments were performed to select the vectorization technique (BoW or TF-IDF), and the text-normalization technique (stemming or lemmatization). These tests used KMEANS with k=5.

Table 14 contains the silhouette scores for each of the different pre-processing strategies tested. The silhouette score was used to compare the different pre-processing steps. Cluster error was not considered, as different normalization techniques yield different geometries.

Another parameter considered is the cumulative explained variance after reduction using TruncatedSVD. Initially, the objective was to assess how many features were needed to capture at least 75% of the variance, with a maximum of 1000 features. However none of the tested combinations led to 75% of explained variance using less than 1000 features.

Among the tested pre-processing strategies, stemming combined with Bag-of-Words (BoW) produced the strongest performance on both metrics. The low silhouette score (0.217) is likely due to high dimensionality, but the high dimensional representation is retained to preserve semantic richness.

An additional pre-processing step applied in this clustering task is the segmentation of comments, exceeding 300 words, into multiple shorter samples. This step was introduced based on empirical observations indicating that cluster assignments are biased by comment length. Figure 6 shows boxplots of the number of words in comments in each cluster. Specifically, when long comments are left unmodified, the comments longer than 312 words are assigned to the same cluster (cluster 1), while another cluster (cluster 2) tends to contain short comments (75th percentile = 13 words). After applying the segmentation step, the distribution of comment lengths across clusters becomes more balanced, although one cluster continues to contain a higher proportion of shorter comments.

A diagram summarizing the steps taken in the pre-processing for this task can be found in the appendix (Figure 20).

Table 14. Comparison of silhouette scores TF-IDF and BoW with Stemming and Lemmization

Vectorization technique	**Text normalization technique**	**Silhouette score**	**Feature reduction**	
			variance explained	components
TF-IDF	Stemming	-2.93×10^{-3}	51%	1000
	Lemmization	-6.19×10^{-3}	40%	1000
BoW	**Stemming**	**0.217**	**69%**	**1000**
	Lemmization	0.197	58%	1000

4.1.2 Determining the number of clusters. For each number of clusters tested, the silhouette score was computed. Due to the high computational cost, the score was not evaluated on the full dataset, instead, three random subsets of 10,000 samples each were used. Although this does not yield an exact value, it provides a reliable approximation of the silhouette score trend as a function of the number of clusters.



(a) Not splitted comments

(b) Splitted comments (300 words maximum)

Fig. 6. Boxplots of words per comment in each cluster.

Table 16. Number of comments contained in each cluster for KMEANS k=3, max_iter=100

Cluster label	Number of comments	Percentage
0	57619	7.8%
1	5288	0.7%
2	676437	91.5%

The parameters used to KMEANS are reported in Table 15. The clustering error and silhouette scores for the combination of parameters tested (varying max_iteration and n_clusters) can be found in figure 7.

With *max_iter*=2 both clustering errors and silhouette scores are significantly higher. This is likely because cluster assignments remain strongly influenced by the initial centroid positions after only two iterations. Therefore *max_iter*=2 was not considered in the choice of the final configuration. The remaining values of *max_iter* yield comparable results, suggesting that cluster assignments stabilize beyond a certain number of iterations.

The choice of number of clusters was based on the silhouette score and the best silhouette score is achieved for 3 clusters. A value of *max_iter* equal to 100 was selected although only minor differences were expected relative to *max_iter*=10 or 50

Table 15. Parameters passed to KMEANS function

init	random_state	n_init	max_iter	tol	n_clusters
random	42	'auto'	[2, 10, 50, **100**]	1e-4	[**3**,4,5,6,7,8,9]

4.1.3 User Distribution and cluster Analysis. The dimensions of the clusters are reported in Table 16. One of the clusters is very big (containing more than 90% of the samples) and one is very small (containing less than 1% of the samples). The cluster sizes being very different from each other is not a unique characteristic of the number of cluster chosen, but can be found for any other value of k. This trend is likely related to the high dimesionality and sparsity of the feature space which can make the identification of well-separated groups difficult.

To analyze the distribution of users across clusters, the ECDF of the number of clusters assigned to each user was plotted (figure 8a), along with the ECDF of the number of comments per user. The number of comments per user was reported to better contextualize the cluster-distribution metric: if an author has only one comment in

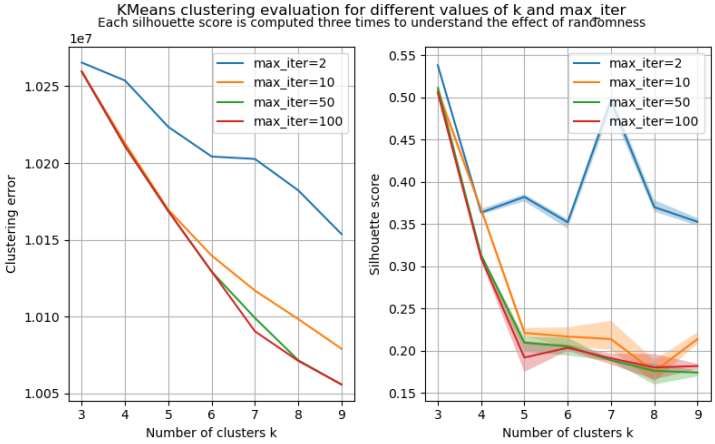
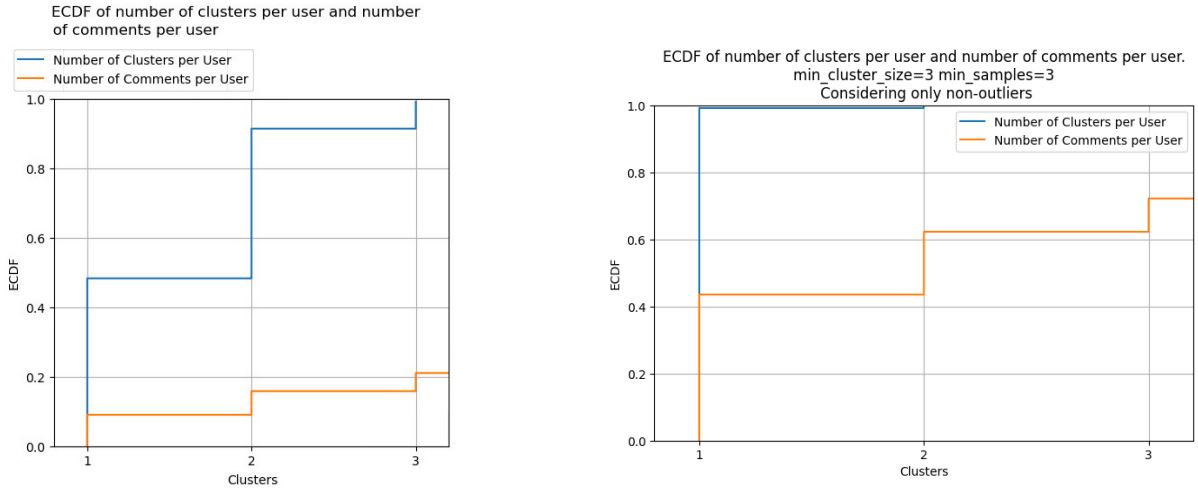


Fig. 7. Clustering errors and silhouette score for number of cluster and max iterations

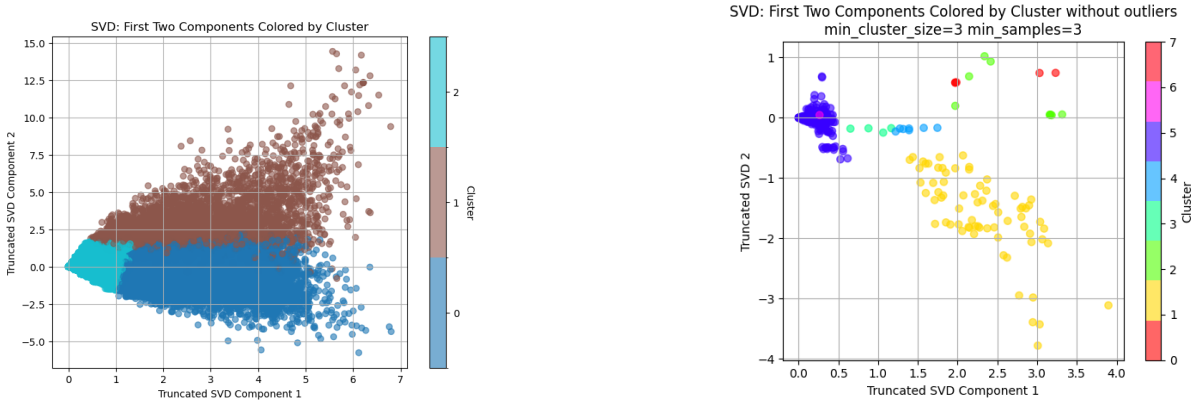
the dataset, then that user can only be assigned to a single cluster. Approximately half of the authors are assigned to a single cluster, while more than 90% are assigned to at most two clusters. The fact that one of the clusters contains less than 1% of the comments likely influenced the ECDF's trend.



(a) KMEANS (b) HDBSCAN

Fig. 8. ECDF of number of clusters per user and number of comments per user.

Figure 9a shows how the comments are distributed on the first two components of Truncated SVD, each comment is colored according to the cluster to which KMEANS assigned it. The cluster with the highest cardinality (cluster number 2) is the closest to the origin. This may be related to the sparse nature of the BoW representation, although the exact position in the reduced space is determined by the TruncatedSVD projection. The clusters are not separated among these two dimensions, and there is also some overlapping.



(a) KMEANS (k=3).

(b) HDBSCAN only non-outliers

Fig. 9. First two components of Truncated SVD, colored by clusters labels

In order to analyze the characteristics of each cluster, TF-IDF scores were used to identify the most representative words of each cluster. Word clouds were used to have a visual representation. Figure 10 shows the most representative words contained in cluster 1. We can see that many of the words present belong to the same semantic area including man, woman, guy, girl, gender, sex. Note that in order to have full words and not just their "base form" the word clouds were obtained using the *non-stemmed* version of the dataset, however this means that both singular and plural can be present in the Word Cloud.

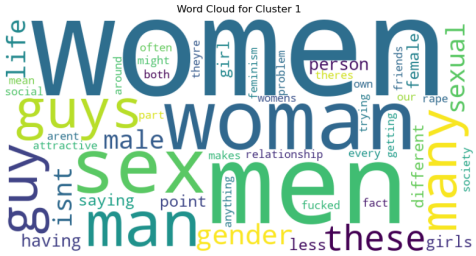


Fig. 10. Word cloud of the most representative words contained in cluster 1 (KMEANS K=3).

In order to understand if clusters reflect subreddit division we used the Adjusted Mutual Information (AMI). The resulting AMI score is 0.01, indicating negligible agreement between the two partitions. This low value suggests that the cluster structure does not meaningfully recover the subreddit-based grouping of the data. AMI is used as the primary metric as it adjusts for chance and is robust to imbalances in label cardinality, making it particularly suitable for this setting compared with NMI or ARI.

4.2 HDBSCAN

4.2.1 Sampling. Due to the computational cost, HDBSCAN could not be executed on the whole dataset. Instead, a meaningful sample of approximately 20800 comments was extracted. To make sure that the subset was meaningful, sampling was performed proportionally from each subreddit, ensuring at least one sample per subreddit. For

Table 17. HDBSCAN: hyperparameters tested, silhouette scores, means cluster persistences

min_cluster_size	min_samples	silhouette score	mean cluster persistence	Outliers
6	6	0.35	0.16	20330
6	3	0.34	0.13	20310
3	3	0.36	0.12	20309

Table 18. HDBSCAN cluster sizes ($min_cluster_size = 3$ and $min_samples = 3$)

Cluster ID	Outliers	5	1	2	4	7	0	6	3
Size	20309	398	71	9	7	5	5	4	4

each subreddit, the longest comment was selected. Since sampling was performed after the segmentation of long comments, most of the comments are likely to be close to 300 words in length.

Figure 20b summarizes the pre-processing pipeline adopted for HDBSCAN. The only difference with the pipeline adopted for KMEANS is the sampling step.

4.2.2 Hyperparameter selection. Testing with $min_cluster_size = 15$ and $min_samples = 5$ HDBSCAN returned a very high number of outliers, therefore, only very small values were considered for both parameters. For each combination of hyper-parameters tested in Table 17 silhouette score was reported (computed excluding the outliers), mean persistence and number of outliers. The tested HDBSCAN configurations yield only moderate silhouette scores and low cluster persistence, indicating weak and unstable cluster structures. Notably, all configurations classify a very large proportion of the data as outliers (more than 97%). No configuration clearly outperformed the others in terms of silhouette score and persistence, hence all three combinations were analyzed to understand if the cluster division identified any group of comments with similar words. The analysis revealed that the most interesting cluster division is the one with the smallest parameters ($min_cluster_size = 3$ and $min_samples = 3$). That is because a smaller $min_cluster_size$ may allow for low-cardinality clusters, in which it is easier to spot semantic relationships between subreddits. Due to report length constraints, only this cluster division will be reported here.

4.2.3 User Distribution and cluster analysis. Table 18 contains the size of each cluster. It should be noted that more than 97% of the samples are classified as outliers. This indicates that even if the division into clusters is semantically meaningful, only a very small subsample of the dataset was clustered. HDBSCAN returned 8 clusters, six of which contain less than 10 comments, while the other two are significantly bigger.

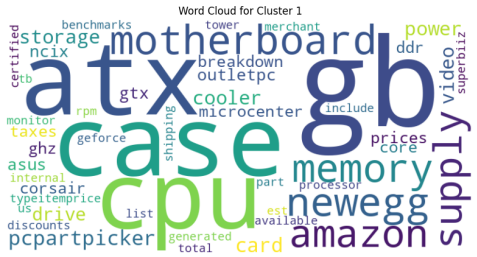
Figure 8b shows the ECDF of the number of clusters assigned to each user, along with the number of comments per user. Almost every author is assigned to one single cluster. Considering that there are 8 clusters, this suggests a more coherent clustering structure compared to KMEANS. However it is important to note that this result is again biased by the big percentage of outliers, the ECDF only considers the authors that have been assigned to clusters, more than 40% of those have only one comment, while when working on the full dataset this percentage was around 10%.

The first two components of the TruncatedSVD of the samples are plotted in figure 9b. For clarity only non-outliers are shown. Comparing the axis labels from this graph with those of KMEANS (figure 9a), it is possible to observe that the samples clustered (the "non-outliers") occupy only a portion of the region defined by Truncanted SVD, the samples outside this region were labeled as outliers. Almost all samples with a value greater than 1 on the second component were labeled as outliers The clusters show different densities, in particular, cluster number

5, the one closest to the origin, is significantly more dense than the other big cluster (cluster number 1). The graph shows that some clusters are well separated, while others partially overlap. This is because inspection of only the first two components does not fully reflect the density relationships used by HDBSCAN in the original feature space (which has 1000 dimensions).

Similarly to what was done for KMEANS, TF-IDF and word cloud were used to gain insights on the characteristics of the clusters. Figure 11 contains the word clouds from the two biggest clusters (clusters 1 and 5) and two other clusters that we thought were interesting (clusters 3 and 4). The word cloud from cluster 1 (Figure 11a) contains many terms related with computers like: case, CPU, GB, atx (Advanced Technology eXtended), motherboard, storage, memory, supply. Cluster 3 (Figure 11b) contains 4 comments from *r/NHL* (National Hockey League), its word cloud contains team-related subreddit names (e.g. r/habs, r/leafs), and player names. The cluster also contains concatenated strings such as "dateawayhometimenetwork", likely originating from scraped tables containing sport statistics. This suggests HDBSCAN can distinguish formatted from conversational text despite preprocessing. The word cloud from cluster 4 (Figure 11c) seems to be related to two different semantic areas, one about medical conditions (hospital, survive, tumor, infection, surgeries, brain) and the other about accademic achievement (graduation, diploma, ceremony). Inspection of the comments revealed that all of them are from the same user (searching on reddit revealed from the same post), and a big portion of the text was identical in all comments. Cluster 5 (Figure 11d) is the largest cluster. Its vocabulary appear "more generic" compared to the ones in the other word clouds however, by looking at all of the words we can see that many of these are related to media/content sharing (post, picture, photo) and positive feedback (nice, awesome, favorite).

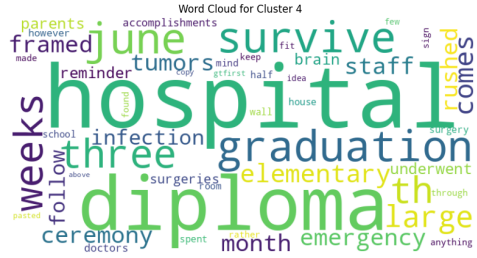
Overall, the word clouds from clusters 3 and 4 suggest that HDBSCAN used on top of BoW can identify highly structured text and repeated patterns.



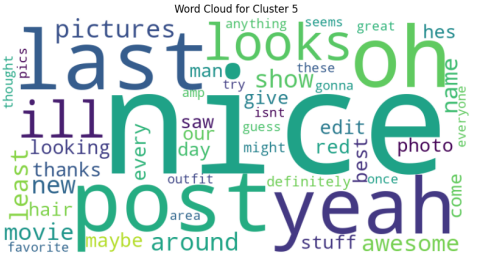
(a) Word cloud for cluster 1



(b) Word cloud for cluster 3



(c) Word cloud for cluster 4



(d) Word cloud cluster 5

Fig. 11. Word clouds from some of the HDBSCAN clusters

Out of the 8 clusters, 3 contain comments from one single subreddit (two of which were previously analyzed), all three have fewer than 10 samples. Cluster 1 contains 71 comments from 5 subreddits. As expected from

the previously analyzed word cloud in Figure 11a, these subreddits are related to computers. Three subreddits: *r/pcmasterrace*, *r/buildapc*, *r/htpc* (Home Theater Personal Computer) focus on computer-related topics, while the other two are about two video games, *r/metalgearsolid* and *r/starcitizen*, where it is probable to find comments about computer.

Figure 12 shows the ECDF of the number of clusters per subreddit and number of comments per subreddit considering only the subreddits that have more than one comment. The ECDF shows that approximately 95% of the subreddits with multiple comments are assigned to a single cluster. This is a very good statistic. However, the large number of unclustered outliers must always be considered.

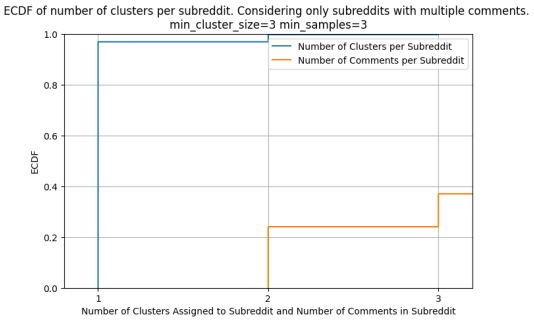


Fig. 12. ECDF of number of clusters per subreddit and number of comments per subreddit. Only subreddits with multiple comments were considered.

4.2.4 Final considerations on the clustering task. KMEANS identified broad structures in the data but produced highly imbalanced clusters, with more than 90% of comments concentrated in a single cluster. Although some semantic relationships emerged through word cloud analysis, the low AMI score demonstrated that the clustering structure did not meaningfully correspond to subreddit divisions.

HDBSCAN provided a complementary perspective. Unlike KMEANS, it was capable of identifying highly specific and semantically coherent groups of comments, particularly clusters containing repeated structures, technical discussions, or subreddit-specific terminology. However, this came at the cost of labeling more than 97% of the samples as outliers. The results indicate that density-based clustering methods can isolate niche semantic patterns, but the dataset overall does not exhibit a strong global density structure.

An important limitation encountered throughout this task is related to the nature of Bag-of-Words representations and, more generally, high-dimensional sparse textual data. Even after dimensionality reduction with Truncated SVD or PCA, the feature space remains extremely large and sparse. In high-dimensional spaces, distances between samples become less informative, making clustering algorithms such as KMEANS less effective at identifying clearly separated groups. This phenomenon contributes to the strong cluster overlap observed in the visualizations and to the tendency of HDBSCAN to classify most points as outliers.

5 Sentiment Analysis

5.1 Sentiment Analysis (classical)

In this section, a pre-trained sentiment analysis model was used to classify some of the comments from the supervised dataset as positive, negative, or neutral. The model chosen for this task was *twitter-roberta-base-sentiment* [1]. It is a model based on *roBERTa* trained on 58M tweets and fine-tuned for sentiment analysis.

The pre-processing differs slightly with respect to the previous sections. For sentiment analysis, removing stopwords may be harmful; therefore, this step was not performed. The other differences with the pre-processing pipeline used in previous tasks are related to the fact that twitter-roberta-base-sentiment takes tokens as input, hence the use of a tokenizer instead of lemmatizaton/stemming and TF-IDF/BoW. For efficiency purposes, the maximum length for the comments was set to 256 characters, and longer comments were truncated by the tokenizer.

Each comment was classified using twitter-roberta-base-sentiment, which outputs three logits corresponding to sentiment classes. These logits were converted into class probabilities using a softmax function.

5.1.1 Overall sentiment distribution across all comments. The overall sentiment distribution across all comments is shown in Figure 13. Most of the comments are labeled as neutral. This class has a mean score of 41.8%. The negative class has a mean score of 34.0%, but the 75th percentile is very close to the one of the neutral class. The positive sentiment is the least represented with a mean score of 24.1%.

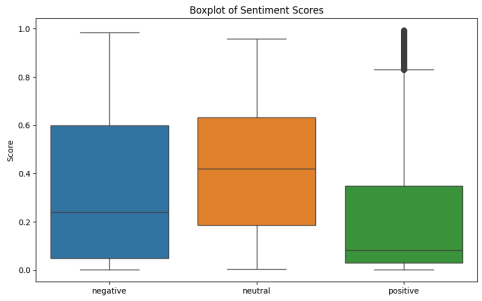


Fig. 13. Boxplot of sentiment scores returned by twitter-roberta-base-sentiment

5.1.2 Sentiment distribution per subreddit. To analyze the sentiment distribution per subreddit, the average score for each sentiment was computed for each subreddit. For each sentiment, the 20 subreddits with the highest mean score were considered and out of these 60 subreddits only 6 have more than one comment. Figure 14 plots the sentiment mean scores for the 20 subreddits with the highest number of comments. Most subreddits in the plot have the highest mean score in the neutral class, with only 4 subreddits showing "negative" as the dominant sentiment, and only one having "positive" as the dominant sentiment. All sentiment scores fall within the range 10% to 60%. Of all values in the plot, only two mean scores exceed 50%, both of which belong to the neutral class. This suggests that, when a large quantity of comments is considered within a subreddit, multiple sentiments tend to coexist rather than having one sentiment prevailing.

5.1.3 Gender sentiment correlation. Figure 15 contains the boxplots of the sentiment scores by gender. There is no clear gender difference in negative sentiment. Female users appear to have slightly higher positive sentiment scores and somewhat lower neutrality compared to male users, suggesting a tendency toward more positive comments.

5.2 Integration challenge

In this section, the sentiment features were combined with the supervised learning pipeline, to understand if gender prediction could be improved using these features. The best model from section 3 was trained again (using the same parameter grid) with the same pre-processing pipeline but this time the sentiment features were also concatenated. The accuracy did not improve, scoring 0.76. This is explainable by considering the number of

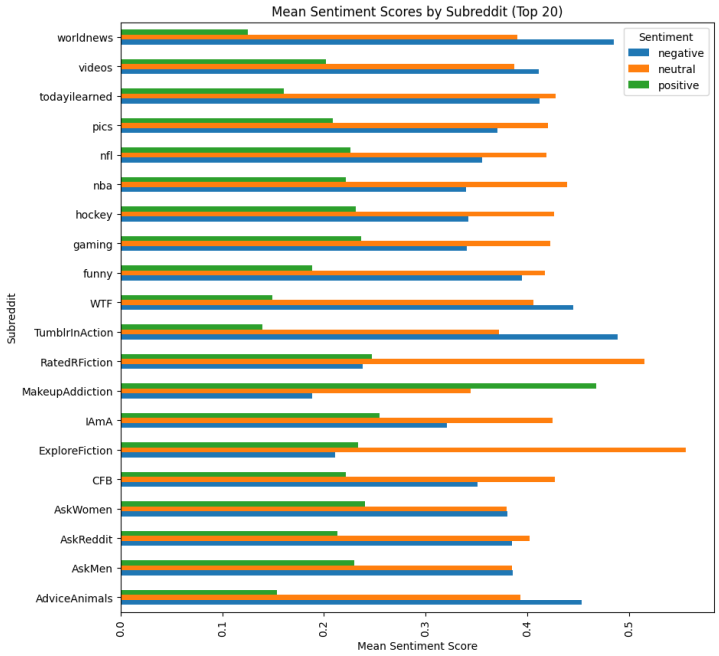


Fig. 14. Bar plot of the mean sentiment scores by subreddits (20 most common subreddits only)

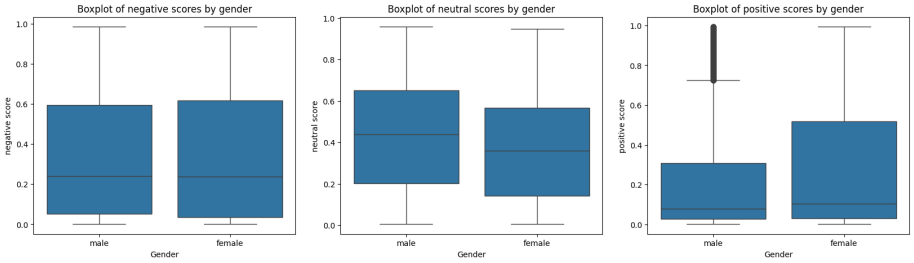


Fig. 15. Boxplot of sentiment scores per gender

features, adding only three features to a dataset that already contains 5000 features is unlikely to provide any significant improvement.

5.3 Sentiment analysis - LLM zero-shot

The purpose of this section is to make a comparison between Large Language Model (LLM) zero-shot classification, manual labeling, and the classification performed in section 5.1. The LLM used for zero-shot classification was *qwen3:1.7b* [4]. Since both manual labeling and LLM-based classification are time consuming tasks, only a subset of 50 comments was classified. The 50 comments were chosen at random, sentiment models often have domain bias (e.g. some subreddits may be sarcasm-heavy while others may be more straightforward), and sampling from one subreddit may increase that domain bias.

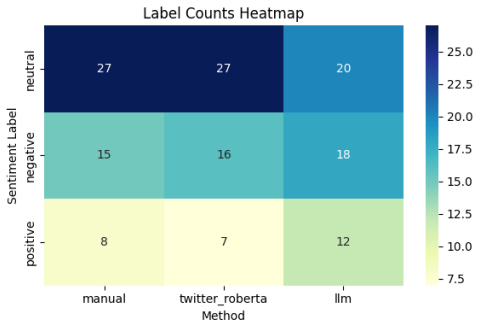


Fig. 16. Number of comments assigned to each label by the three classification methods

Figure 16 presents the number of comments assigned to each label by the three classification methods. The label counts from twitter-roberta-base-sentiment are very close to the ones from manual labeling, while the LLM seems to have classified fewer comments as neutral and more comments as positive. Another metric used for comparison is the number of comments classified differently with respect to the manually labeled comments. The classification from twitter-roberta-base-sentiment agrees more with the human-created labels, with 12 disagreements, while the LLM disagrees in 19 comments.

Based on this small scale experiment, the LLM performs worse than twitter-roberta-base-sentiment. This is expected as the latter was trained with the goal of predicting sentiments, while LLMs are designed to predict the next word in a sequence. However, it is important to note that model size may also affect this, a larger LLM model may perform better than the 1.7-billion-parameter model used. Another aspect to take into consideration is that there are no ground-truth labels for the comments, the manual labels were obtained by reading the comments, but this approach does not consider the context in which the comment was posted.

6 Conclusion

This project explored the use of machine learning techniques to analyze Reddit comments and identify meaningful patterns in user-generated textual data. In the supervised task, several models were tested for gender prediction: Linear SVM, Random Forest and Logistic Regression. The best results were obtained by combining textual features with subreddit information using random forest classifier, highlighting the importance of user activity patterns.

The unsupervised analysis highlighted the difficulty of clustering high-dimensional and sparse textual data. K-Means was able to identify broad structures, but the resulting clusters were highly imbalanced and only weakly related to subreddit categories. HDBSCAN, on the other hand, detected smaller and more semantically coherent groups, but classified most samples as outliers, suggesting that the dataset does not contain a strong global density-based cluster structure.

Finally, sentiment analysis showed that Reddit comments are mostly neutral, with positive and negative sentiments coexisting across communities. However, adding sentiment features did not significantly improve gender classification, likely because sentiment information was too limited compared to the large number of textual features.

Overall, the results suggest that Reddit comments contain useful linguistic and behavioral signals for user profiling, but also reveal important challenges related to class imbalance, sparse representations, overfitting, and the complexity of informal online language. Future work could improve the analysis by using more advanced language models, better class-balancing strategies, richer behavioral features, and more scalable clustering methods designed for high-dimensional textual data.

References

[1] Francesco Barbieri, Jose Camacho-Collados, Leonardo Neves, and Luis Espinosa-Anke. 2020. TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification. arXiv:2010.12421 [cs.CL] https://arxiv.org/abs/2010.12421

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR* abs/1810.04805 (2018). arXiv:1810.04805 http://arxiv.org/abs/1810.04805

[3] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *CoRR* abs/1910.01108 (2019). arXiv:1910.01108 http://arxiv.org/abs/1910.01108

[4] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

A Language Models Exploration

The objective of this section is to harness the capabilities of a language model like BERT [2], for supervised learning with the same goal of section 3 (predicting users' gender) but using a different representation for the comments.

A.1 BERT

BERT stands for Bidirectional Encoder Representations from Transformers. It is based on the Transformer architecture, which uses a mechanism called self-attention to model relationships between all words in a sentence simultaneously. Traditional language models process text either left-to-right or right-to-left. In contrast, BERT is bidirectional, meaning it looks at the entire sentence at once, considering both the left and right context of each word. This allows BERT to learn deeper and more context-aware representations.

BERT was used to extract embeddings from each comment. Embeddings are numerical representations that capture the meaning of words or phrases in a continuous, high-dimensional space. Unlike Bag-of-Words or TF-IDF, which represent words based on frequency and ignore context, BERT's embeddings capture rich semantic and contextual meaning, allowing the same word to have different representations depending on its surrounding words. The version used is *DistilBERT* [3], which is the small and fast version of BERT.

A.2 Feature extraction

We worked on the supervised dataset grouping the comments by authors. For each user, we created a vector that represents the number of comments written in each subreddit. We then reduce the cardinality of this with PCA.

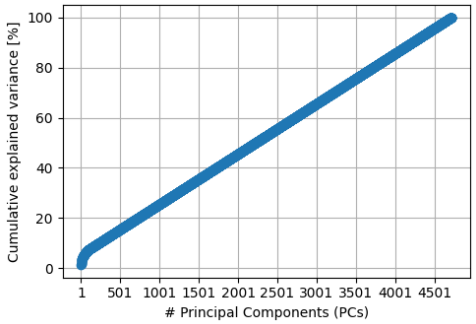


Fig. 17. Percentage of cumulative explained variance per number of principal components

We wanted to keep at least 75% of the variance, based on the cumulative explained variance curve, we choose to retain 3500 features resulting in a cumulative explained variance of 76.5% . We then concatenated the embeddings obtained from *distilBERT* to the results of the PCA. The project guideline required the use of the concatenated data to perform classification. However, since DistilBERT embeddings contain only 768 features, substantially fewer than the PCA-transformed subreddit features, we decided to also test how the model would perform using only the embeddings as input.

A.3 Model fine tuning

The multilayer perceptron was built using *torch*. The model consists of an input layer, two or three hidden layers, and an output layer. Each hidden layer is followed by an activation function. The activation function introduces non-linearity and allows the network to learn complex feature representations. We evaluated both hyperbolic

tangent (tanh) and LeakyReLU activations. The output layer contains two neurons, corresponding to the two target classes.

Cross-Entropy Loss was used as the loss function, while *Adam* was adopted as the optimizer.

In order to choose the hyper-parameters we define a parameter grid (Table 19).

Table 19. Grids of hyperparameters explored during model tuning for the two layers and three layers models.

Hyperparameter	**Values Tested, 2 layers model**	**Values Tested, 3 layers model**
Learning rate	1e-5	1e-5
Hidden layer 1	7500, 5000, 2000	4000, 2000, 1000
Hidden layer 2	5000, 2500, 1250	2500,1500, 750
Hidden layer 3	-	250,500,1000
Batch size	32,64	32,64
Activation function	Tanh	Tanh, LeakyReLu

We trained a model for each combination of the hyper-parameters using a training loop without a fixed number of epochs, early stopping was implemented using a patience parameter (number of epochs with no improvement after which training will be stopped). After each training epoch, we made the model predict the gender of each sample of the validation data set. The model configuration achieving the highest validation accuracy across all epochs and hyperparameter configurations was selected.

After having selected the hyper-parameters, we trained the model again with the same training loop but this time using a higher patience value. A larger patience value may improve final performance at the expense of longer training time.

We followed this process to train three different models with different activation functions (Tanh and LeakyReLu) and number of hidden layers (2 and 3).

A.3.1 Training results. This subsection contains the performance results from the six models trained. We trained six models in total — three on embeddings alone and three on concatenated features — varying depth (2 vs. 3 layers) and activation function (tanh vs. LeakyReLU).

For each model, we plotted the evolution of the training and validation losses and accuracies across epochs to assess convergence and possible overfitting tendencies. In this report, only the charts for the models that achieved the best performance are included.

The model that performed best (considering the accuracy) when training with just the embeddings was the one with three layers that uses LeakyReLu as activation function. We can see the learning curve in Figure 18. Signs of overfitting appear after epoch 5, the loss on the training keeps decreasing while the one on the validation flattens out, we can also see the overfit from the accuracy curves, the accuracy on training continues to increase, whereas the accuracy on the validation slightly decreases. The training procedure stored the model parameters corresponding to the highest validation accuracy, so after training it restored the state of the model at epoch 5. The model with the same architecture is also the one that performed the best when using the concatenated data for training. We can see the learning curves in Figure 19. The model had the highest validation accuracy at epoch 10, we can see from the loss curve that at that epoch some overfitting is already observable since the training curve has a significantly lower value than the validation curve, however, the value of validation loss at epoch 10 is close to the minimum of the curve. Comparing these graphs with those obtained from the model trained using only the embeddings we can see that when using the concatenated data the model is more prone to overfitting. This behavior may be related to the increase of the number of features that make the problem too complex for the number of neurons in the neural network. In Table 20 we reported for each of the model trained the accuracy,

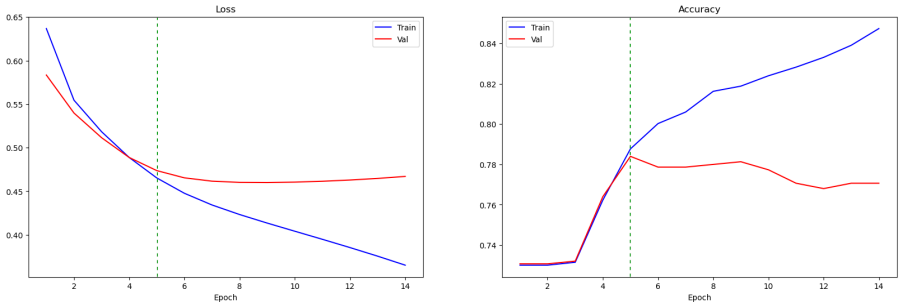


Fig. 18. Learning curves for the model with 3 layers that uses LeakyReLu trained with just the embedding. Dashed line: epoch with highest validation accuracy (restored checkpoint).

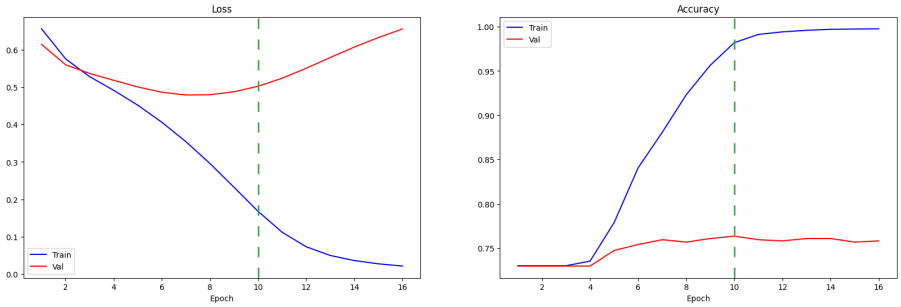


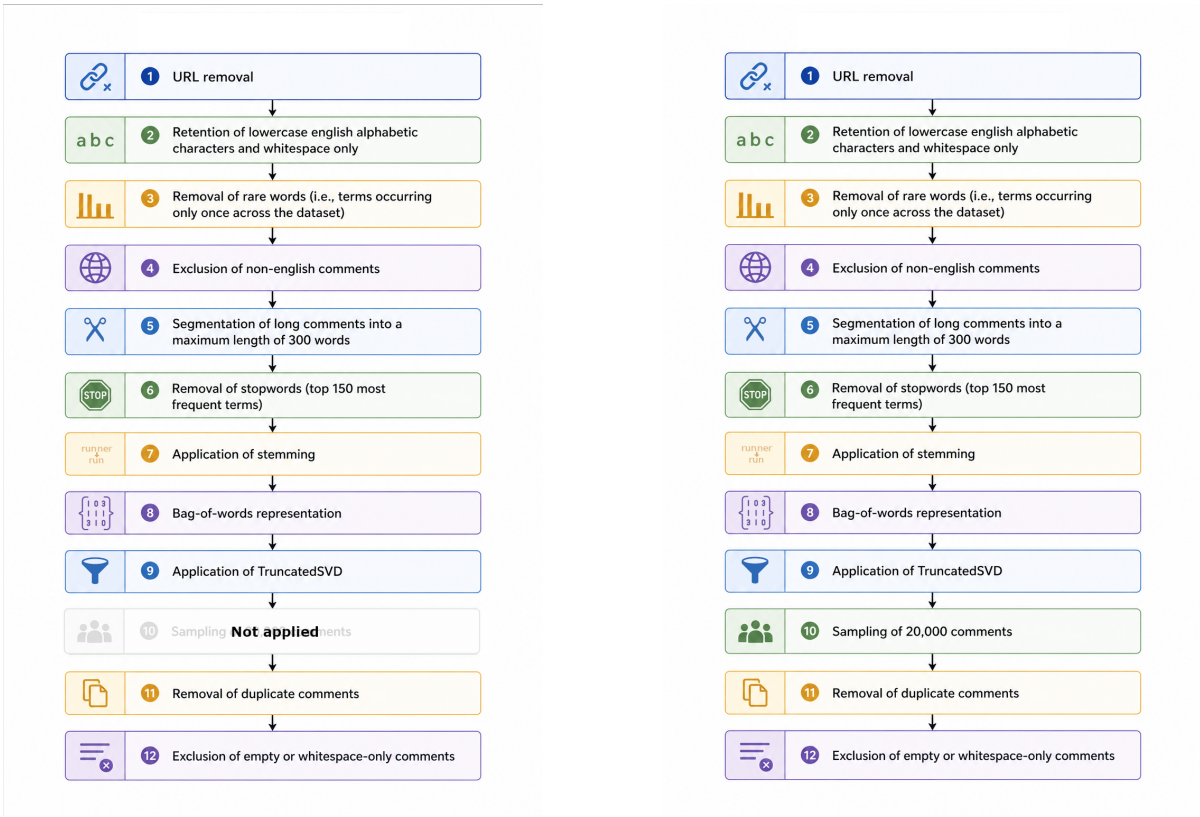
Fig. 19. Learning curves for the model with 3 layers that uses LeakyReLu trained with the concatenated data. Dashed line: epoch with highest validation accuracy (restored checkpoint).

recall and F1-score on validation set. Comparing the architectures, we can see that the models with three hidden layers perform better than the ones with two. This suggests that the models with only two hidden layers are too simple for this task. Regarding the activation function, LeakyReLu outperforms tanh when training using the "concatenated data" but they perform similarly when training using only the embeddings obtained from BERT. The results also show that models performed better when trained using just the embeddings, this may be due to the difference in the number of feature, even if the information about how much a user has posted in a given subreddit might be relevant since it is highly sparse; the model may struggle to learn informative representations from these sparse inputs.

Table 20. Model performance comparison for different data and architectures

Data used in training	# of layers	Activation function	Accuracy	Precision	Recall	F1-score
Just embeddings	3	tanh	0.783	**0.778**	0.783	**0.780**
	2	tanh	0.769	0.762	0.769	0.765
	3	LeakyReLu	**0.784**	0.773	**0.784**	0.753
Concatenated data	3	tanh	0.749	0.749	0.749	0.749
	2	tanh	0.741	0.761	0.741	0.748
	3	LeakyReLu	0.764	0.744	0.764	0.745

B Summary of the pre-processing steps taken for unsupervised learning



(a) KMEANS (b) HDBSCAN

Fig. 20. Pre-processing pipeline for unsupervised learning